The Lighten Road - GDD System Choice

DGD3004 Game Development I | Homework 2 (Part B)

Team: Furkan Aydın, Siraç Göktuğ Şimşek, Can Mutluşahinoğlu, Maksim Martynenko, Kemal Berke Aşık

B.1 The system

Field	From Game_Systems_Implementations.xlsx
System name	Lore Collectibles / Codex
Tab	World & Discovery
How It Can Be Implemented	Items, books, and audio logs populate an in-game encyclopedia.
Example Games	Mass Effect Codex, Horizon datapoints, Destiny Grimoire

B.2 Why this system, for whom

This system primarily serves our Explorer players, who search for hidden Old World fragments and the truth behind the Fall. It also rewards Achievers who manage their lantern economy well enough to take risky optional detours. It connects Step 3 of our core loop, Reward (overcome the obstacle, uncover lore and scavenge ember fuel), with Step 4, New decision (continue to the exit or detour for more collectibles). The player should feel tension between fear of darkness and curiosity, followed by relief when a recovered log reveals a useful clue and restores one ember.

B.3 How it works in your game

What the player does. The player starts each chamber with three embers and can carry at most four. Dim mode provides passive light while at least one ember remains; a Flare permanently spends one ember for that chamber to reveal a wider area. The player can detour to an Archive Terminal, interact with it and read its log in the Codex. Powering an Ember Vein temporarily stores one ember in machinery; draining it returns that ember, so the player weighs access to a hidden record against the remaining fuel.

What the game tracks. Each log has a unique ID. The game records discovered IDs, Archive Decryption % (unique logs found divided by unique logs in the catalog), and whether each delivered event is live or a replay (isReplay). Different terminals pointing to the same log still count as one discovery.

What changes. A newly discovered log opens a page containing lore and a room clue, and grants one emergency ember, capped at four held embers. Reading the same log again only reopens its page; it never grants another reward. More than three seconds at zero embers causes Darkness death and a chamber reset with three embers. Discovered logs survive death and scene reloads for the rest of the play session; a new play session starts a fresh Codex.

Observer path. A terminal's DiegeticWhisper requests discovery through CodexSession, which validates the ID and raises SocialChannel.Raise(e) with kind LogFound and the ID in text. StoredDelivery keeps the event and marks copies isReplay = true when a listener subscribes later. The Codex UI rebuilds its entries from replay, while LanternFuel ignores replayed events. The terminal has no reference to either listener or to the recovery feed.

Concrete play moment - illustrative playtest-style scenario

1. The player enters Chamber 2 in Dim mode with 3/4 embers.
2. A Flare reveals an upper Archive Terminal; the ember count falls from 3 to 2.
3. The player powers a shelf vein to reach the detour, leaving 1 ember in the lantern.
4. The player crosses the dim route and reaches a second, short bridge circuit.
5. Powering that circuit leaves 0 embers and starts the three-second Darkness countdown.
6. Before time runs out, the player reads a new log; its live event restores 1 ember.
7. The countdown stops; the player reads the clue, then drains both veins to reach 3 embers.
8. The player exits; after a death or reload, that saved log reappears without another reward.

The Unity side

Piece	Role and designer-facing data
Game scripts	DiegeticWhisper handles terminal input. CodexSession validates and remembers discoveries. ArchiveCodexMenu implements ISocialListener, builds the Codex and shows decryption progress. The existing static ArchiveMenuStyle supplies typography; it is not a listener.
Fuel and room lifecycle	LanternFuel implements ISocialListener and adds one ember on a new live log only. LanternController handles Dim/Flare. PlayerRespawnController handles Darkness death and chamber reset without clearing Codex history. Existing vein, shelf and water scripts remain the environmental costs of a detour.
Lecture scripts reused	From 03_Observer: SocialChannel, ISocialListener, SocialEvent, SocialEventKind, StoredDelivery, PartyFeedUI, DeliveryPolicy, MembershipPolicy and AmbientMembership, with their PartyMember/PartyRole dependencies. Add LogFound to the enum and a formatting case to SocialEvent.ToString; keep the lecture delivery and membership logic unchanged.
Designer assets	One SocialChannel preset with its own StoredDelivery and AmbientMembership assets; an ArchiveCatalog; ArchiveLog assets with ID, title, text, room clue and optional audio; an ArchiveTerminal prefab and scene placements. Designers tune the log content and layout while keeping IDs unique.

Replay and presentation. ArchiveCodexMenu clears its local list before subscribing, then reconstructs pages from stored events. PartyFeedUI formats recovery lines; a separate visibility listener displays live notifications briefly. LanternFuel returns immediately for isReplay events, preventing repeat rewards after re-enabling listeners or loading another scene. Runtime copies of the channel and delivery policy preserve history within a session without turning the asset into a permanent save. Opening the Codex pauses simulation and gameplay input; closing it restores them.

Authorship and AI use. The team selected Lore Collectibles / Codex and defined the core rules, including the first-discovery ember reward. AI assisted English wording, document organization, technical consistency checks and the Unity implementation of that team-defined system. The prototype in-game log prose was also AI-assisted.

Source. Game_Systems_Implementations.xlsx, World & Discovery, Lore Collectibles / Codex. The two quoted content cells in B.1 are copied verbatim; the remaining section describes the team's game.